

Git Fundamentals

Git is one of the most important tools used in software development and DevOps. It allows developers to track changes, maintain project history, collaborate safely, experiment with code, and restore older versions when something goes wrong.

1. Why Does Git Exist?

Imagine you are developing a project called:

```
payment-service
```

Over a few days, the project changes:

```
Day 1 → Payment API working  
Day 2 → Added discount logic  
Day 3 → Refactored payment logic  
Day 4 → Everything broke
```

Now suppose you want to return to the version from Day 2.

Without version control, people often start creating copies like:

```
payment-final  
payment-final2  
payment-working  
payment-final-working  
payment-final-final  
payment-final-final-2
```

This is basically **manual version control**. It quickly becomes difficult to manage.

The problem becomes even bigger when multiple developers work on the same project.

Example:

```
Aditya → Authentication
Rohit → Payment
Harshita → Frontend
Aman → Orders
Rahul → Notifications
```

Now we need answers to questions such as:

- Who changed a particular line?
- When was it changed?
- Why was it changed?
- What did the project look like yesterday?
- Can multiple developers work at the same time?
- Can someone experiment without breaking stable code?
- Can changes from different developers be combined?
- Can an older version be restored?

These are the problems that a **Version Control System** solves.

2. What Is Version Control?

A Version Control System tracks changes made to files over time so that we can inspect, compare, collaborate on, and restore different versions of a project.

Git is a Version Control System.

```
Version Control System
  ↓
  Git
```

Git maintains the **history of a project**.

Instead of keeping multiple manually copied folders, Git records meaningful versions of the project as commits.

3. Git vs GitHub

Git and GitHub are related, but they are not the same thing.

Git

```
Git
|
├─ Version Control System
├─ Runs on your machine
├─ Tracks project history
└─ Manages commits, branches, merges, etc.
```

GitHub

```
GitHub
|
├─ Cloud platform
├─ Hosts Git repositories
├─ Enables collaboration
├─ Pull Requests
├─ Issues
└─ CI/CD integrations
```

Git can work completely on your local machine without GitHub.

GitHub becomes useful when repositories need to be shared or multiple developers need to collaborate.

```
Developer A
  \
Developer B — GitHub Repository
  /
Developer C
```

Other platforms built around Git include:

GitLab

4. Creating a Git Repository

Create a directory for the project:

```
mkdir git-masterclass  
cd git-masterclass
```

Create a simple file:

```
echo "Coder Army Application" > app.txt
```

At this point, this is only a normal directory.

If you run:

```
git status
```

Git will report something similar to:

```
fatal: not a git repository
```

The reason is simple:

```
Normal Directory  
  ≠  
Git Repository
```

Git does not automatically track every folder on the system.

5. `git init`

To convert the current directory into a Git repository:

```
git init
```

Now inspect hidden files:

```
ls -la
```

You should see:

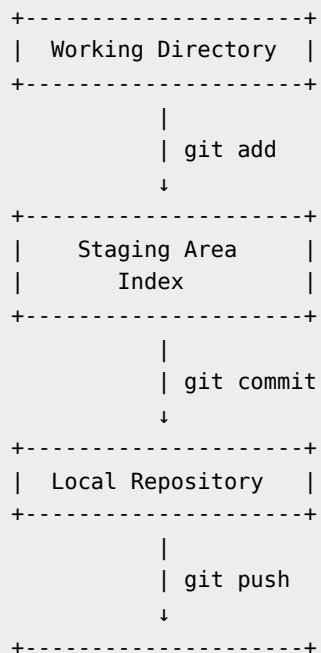
```
.git
```

The hidden `.git` directory is what turns a normal project directory into a Git repository.

The actual repository metadata, objects, references, configuration, and history are stored inside `.git`.

6. The Most Important Git Mental Model

A basic Git workflow can be understood using four areas:



```
| Remote Repository |  
|      GitHub      |  
+-----+
```

The normal flow is:

```
Working Directory  
  ↓ git add  
Staging Area  
  ↓ git commit  
Local Repository  
  ↓ git push  
Remote Repository
```

Understanding these areas makes most Git commands much easier to reason about.

7. Working Directory

The **working directory** is the project you are currently editing.

For example:

```
git-masterclass/  
├─ app.txt  
└─ ...
```

To check the current state of the repository:

```
git status
```

If a new file has not yet been added to Git, it may appear as:

```
Untracked file
```

Two important states for beginners are:

Untracked
Staged

Untracked

An untracked file exists in the working directory, but Git is not currently tracking it as part of the repository history.

Staged

A staged file has been selected for inclusion in the next commit.

Conceptually, staging means:

| Include this version of this file in my next commit.

8. Why Does the Staging Area Exist?

Suppose you changed three files:

```
login.java  
payment.java  
README.md
```

But your next commit should contain only:

```
login.java  
payment.java
```

You can stage only those two files:

```
git add login.java payment.java
```

The `README.md` changes remain in the working directory but are not included in the next commit.

This is why the staging area is useful.

| The staging area allows us to construct the next commit.

It gives us control over exactly which changes belong together.

9. Configuring Git Identity

Before creating commits, Git may require your identity.

Configure it globally:

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
```

To verify the current values:

```
git config user.name
git config user.email
```

The configured name and email become part of commit metadata.

10. Creating the First Commit

After staging the required files, create a commit:

```
git commit -m "Initial commit"
```

Git will create a commit with information conceptually similar to:

```
commit a831c2392...
Author: ...
Date: ...

    Initial commit
```

The long value such as:


```
a831c2392...
```

is the identifier of that commit.

A commit represents a recorded version of the project together with metadata such as the author, timestamp, parent commit, and commit message.

11. Viewing Git History

After creating more commits, Git builds a project history.

Useful commands:

```
git log
```

For a compact view:

```
git log --oneline
```

Conceptually:

```
A ————— B
Initial      Add Login
```

Your project now has a history instead of only its current state.

This history later becomes the foundation for concepts such as branching, merging, and rebasing.

12. Local Repository vs Remote Repository

So far, the complete Git repository exists only on your machine:

```
Laptop
└─ git-masterclass
```

```
└─ .git
```

The `.git` directory contains the local repository data and history.

If that repository exists only on one laptop, collaboration becomes difficult and losing the machine can also mean losing the local repository history.

A remote repository solves this problem.

```
Laptop
Local Repository
    |
    | git push
    ↓
GitHub
Remote Repository
```

13. Adding a Remote Repository

Create an empty repository on GitHub and connect the local repository to it.

```
git remote add origin <repository-url>
```

To inspect configured remotes:

```
git remote -v
```

Example idea:

```
origin → https://github.com/.../git-masterclass.git
```

`origin` is simply the conventional name commonly used for the primary remote repository.

It is a name pointing to a remote repository URL.

14. Pushing Changes

If your current branch is `master`, you can push it using:

```
git push -u origin master
```

Conceptually:

```
Local Repository
  |
  | git push
  v
Remote Repository
```

The `-u` option sets an upstream relationship, allowing future commands such as `git push` and `git pull` to know which remote branch the local branch is associated with.

For a temporary credential cache during a live environment, Git can be configured to keep credentials in memory for a limited period.

Example from the lecture setup:

```
git config --global credential.helper 'cache --timeout=14400'
```

```
14400 seconds = 4 hours
```

15. Cloning a Repository

Suppose another developer joins the project.

They do not have the project files or the `.git` repository.

They can clone the repository:

```
git clone <repository-url>
```

A clone gives them much more than a copy of the visible project files.

Conceptually:

```
Project files
+
Git history
+
Branches
+
Git metadata
+
Remote configuration
```

ZIP vs Clone

Downloading a ZIP mainly gives you:

```
Files
```

Cloning gives you:

```
Files
+
Repository history
+
Branches
+
Git metadata
+
Remote configuration
```

This is why cloning a repository is fundamentally different from downloading its source code as a ZIP file.

16. `git fetch` vs `git pull`

This distinction is extremely important.

Suppose the remote repository contains:

```
A — B — C
```

But your local branch currently contains:

```
A — B
```

Another developer has pushed commit `C`.

```
git fetch
```

Run:

```
git fetch
```

Conceptually, `git fetch` means:

```
Download remote information and objects
without automatically integrating them
into the current local branch.
```

After fetching, the situation might look like:

```
master
↓
B

origin/master
↓
C
```

Your local `master` still points to `B`.

But your local repository now knows that the remote-tracking branch `origin/master` points to `C`.

So:

```
git fetch
→ Bring remote information locally
→ Do not automatically integrate it into the current branch
```

git pull

Run:

```
git pull
```

A useful beginner mental model is:

```
git pull
  ↓
  fetch
  +
  integrate
```

Traditionally, this is often thought of as:

```
git fetch
+
git merge
```

The exact integration behavior can depend on Git configuration and command options.

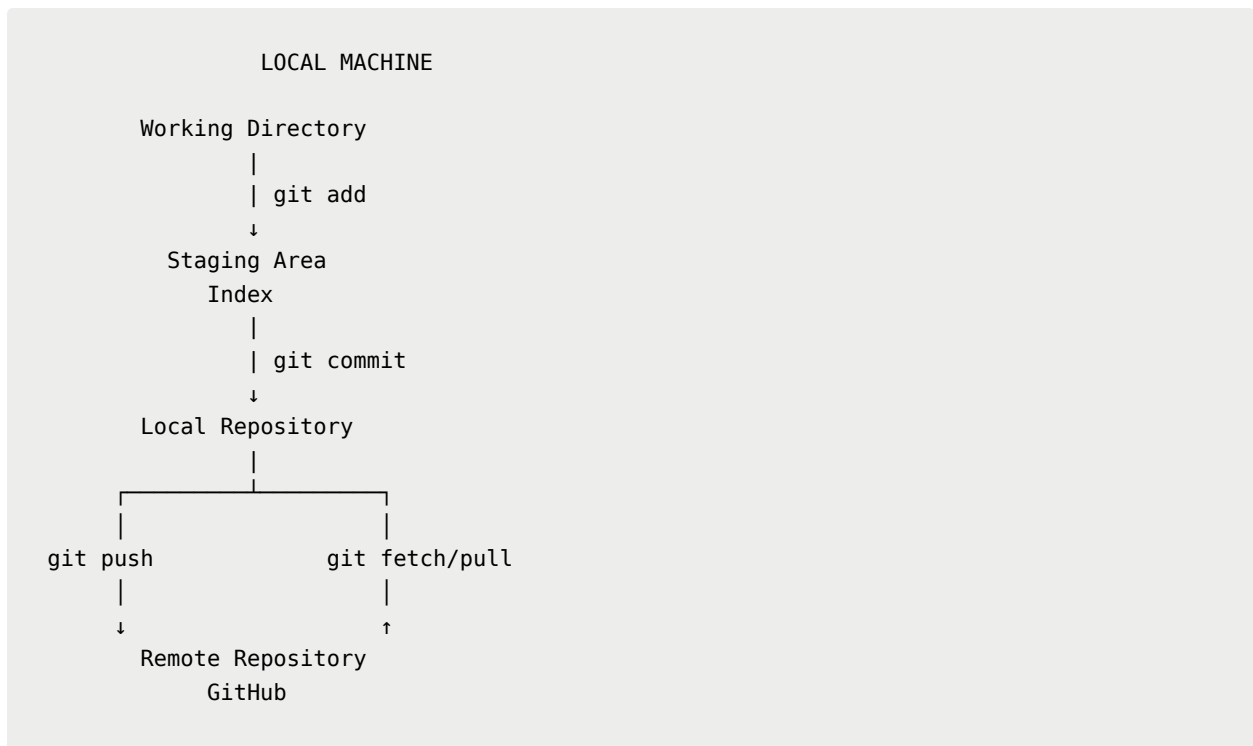
The important distinction is:

```
git fetch
→ Download remote changes/information

git pull
→ Download remote changes and integrate them
```

This becomes especially important when learning merge and rebase.

17. Complete Local and Remote Workflow



18. Git Internals: How Git Actually Works

Git is not simply a collection of commands.

Internally, a Git repository behaves like a database stored inside the `.git` directory.

Inspect it using:

```
ls -la .git
```

Depending on the Git version and repository state, you may see entries such as:

```
HEAD
config
description
hooks/
index
info/
```

```
logs/  
objects/  
refs/
```

The most important parts for understanding Git internals are:

```
.git/  
|  
├─ objects/    → Git object database  
├─ refs/       → Branches, tags, etc.  
├─ HEAD       → Current position  
├─ index       → Staging area  
└─ config      → Repository configuration
```

The Git repository is essentially a local database stored inside `.git`.

19. Git as a Content-Addressed Object Database

Git is fundamentally built around a **content-addressed object database**.

The basic idea is:

```
Content  
  ↓  
Hash  
  ↓  
Object ID
```

Git stores objects identified by a hash derived from the object's content and object metadata.

Conceptually:

```
"Hello Coder Army"  
  ↓  
  hashing
```



```
↓  
abc123...
```

The resulting hash acts as an identifier.

Git can use object IDs to refer to stored content and determine when identical content can be reused.

This is a major part of how Git stores project history efficiently.

20. Core Git Objects Covered Here

The Git internals in this lecture focus on three important object types:

1. Blob
2. Tree
3. Commit

Together, these explain how Git stores file contents, directory structures, project snapshots, and commit history.

21. Blob Objects

Suppose a file exists:

```
hello.txt
```

with the content:

```
Hello Coder Army
```

Git primarily stores the **content** of the file in a blob object.

Conceptually:

```
hello.txt
  |
  ↓
"Hello Coder Army"
  |
  ↓
Blob Object
```

A blob represents file content.

A blob does **not** primarily care about the filename.

22. Inspecting a Blob with `git hash-object`

To calculate the object ID for a file:

```
git hash-object hello.txt
```

Git returns a hash.

Example:

```
d2a84f4b...
```

This command calculates the object ID but does not necessarily write the object into the object database.

To write the blob object:

```
git hash-object -w hello.txt
```

The `-w` option tells Git to write the object into its object database if needed.

Example object ID from the lecture:

```
5da7cf708a27a7b4769256a81332d7ee31bb673d
```

23. Where Git Stores Loose Objects

Git does not normally create one file directly named with the entire hash.

For this object ID:

```
5da7cf708a27a7b4769256a81332d7ee31bb673d
```

Git splits it conceptually as:

```
5d | a7cf708a27a7b4769256a81332d7ee31bb673d
^^  ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
dir  filename
```

The object is stored under:

```
.git/
├─ objects/
│   └─ 5d/
│       └─ a7cf708a27a7b4769256a81332d7ee31bb673d
```

Other directories such as:

```
12
15
4e
79
```

represent the first two hexadecimal characters of other object IDs already present in the repository.

24. Why **cat** Does Not Show the Original Blob Cleanly

If you directly inspect a loose object file with a normal `cat` command, you will not simply see:

```
Hello Coder Army
```

Git does not store a loose object as a normal plaintext copy of the file.

The object is stored using Git's internal compressed representation.

Instead, Git provides commands for inspecting objects.

25. Inspecting Git Objects with `git cat-file`

To check the type of an object:

```
git cat-file -t <hash>
```

Example output:

```
blob
```

To display the object's content:

```
git cat-file -p <hash>
```

Example:

```
Hello Coder Army
```

Mental model:

```
git cat-file -t  
→ What type of Git object is this?
```

```
git cat-file -p  
→ Pretty-print/display the object's contents
```

These are mainly useful for understanding and diagnosing Git internals. They are not commands you need for every normal Git workflow.

26. What Happens Internally During `git add` ?

Normally, you simply run:

```
git add hello.txt
```

From a beginner's point of view:

```
git add  
→ stages a file
```

Internally, there is more happening.

Conceptually:

```
hello.txt  
↓  
calculate the content hash  
↓  
create/write the blob object in .git/objects  
↓  
record the blob reference in the staging area/index
```

So when you run:

```
git add hello.txt
```

Git can store the file content as a blob object and update the index to represent the version intended for the next commit.

27. Same Content → Same Blob ID

Create another file:

```
hello-copy.txt
```

with exactly the same bytes:

```
Hello Coder Army
```

Now run:

```
git hash-object hello.txt
git hash-object hello-copy.txt
```

If the contents are identical, the object IDs will also be identical.

This demonstrates an important principle:

```
Filename does not determine the Blob ID.

Content determines object identity.
```

So two files with different names can refer to the same blob when their contents are identical.

This also helps Git reuse unchanged content.

28. Recovering Content Directly from a Blob

Suppose a blob has already been written:

```
hello.txt
  ↓
git hash-object -w hello.txt
  ↓
5da7cf...
```

```
↓  
.git/objects/5d/a7cf...
```

Now remove the working-directory file:

```
rm hello.txt
```

Verify:

```
ls
```

The visible file is gone, but the Git object can still exist in the object database.

Read it using:

```
git cat-file -p 5da7cf708a27a7b4769256a81332d7ee31bb673d
```

The original content appears.

You can even reconstruct the file:

```
git cat-file -p 5da7cf708a27a7b4769256a81332d7ee31bb673d > hello.txt
```

Then verify:

```
cat hello.txt
```

Conceptually:

```
hello.txt  
  ↓ git hash-object -w  
  
Blob Object  
5da7cf...  
  ↓  
  
.git/objects/  
├─ 5d/  
  └─ a7cf...
```

```
↓  
  
delete hello.txt  
  
Working file gone  
Git object still exists  
↓  
  
git cat-file -p  
↓  
  
Original content recovered
```

This demonstrates that the working-directory file and the object stored inside Git are separate things.

29. Tree Objects

A blob stores file content, but that raises an important question:

If a blob stores the content, where does Git store the filename and directory structure?

That responsibility belongs to **tree objects**.

Conceptually:

```
Tree  
|  
├─ hello.txt ────> Blob A  
├─ app.txt ────> Blob B  
└─ src/ ────> Tree C
```

A tree represents a directory-like structure.

A tree connects information such as:

```
name  
+  
mode/type
```



```
+  
object ID
```

Example:

```
Tree  
↓  
hello.txt → blob abc123  
app.txt   → blob def456  
src       → tree 789xyz
```

The blob knows the content.

The tree gives that content a name and places it inside a directory structure.

30. Inspecting a Tree

You can inspect the tree represented by a commit or branch.

For example:

```
git ls-tree master
```

or:

```
git ls-tree HEAD
```

You may see output such as:

```
100644 blob abc123... app.txt  
100644 blob def456... hello.txt
```

A directory may appear as a tree:

```
040000 tree xyz789... src
```

You can then inspect a tree object directly:

```
git cat-file -p <tree-hash>
```

The key distinction is:

```
Blob
→ file content

Tree
→ names + directory structure + references to objects
```

31. Commit Objects

A commit object points to a **tree** representing the project's snapshot.

Conceptually:

```
Commit C
|
├─ Tree T
|   ├── app.txt → Blob B1
|   └── src/    → Tree T2
|
├─ Parent → Commit B
├─ Author
├─ Committer
├─ Timestamp
└─ Message
```

A commit therefore contains more than only a message.

It connects project content with history and metadata.

32. Inspecting a Commit Object

You can inspect the commit referenced by a branch:

```
git cat-file -p master
```

You may see something conceptually similar to:

```
tree 74fd...
parent 12ab...
author Aditya <...> 178...
committer Aditya <...> 178...

Add login feature
```

The commit points to a tree.

You can inspect that tree:

```
git ls-tree 74fd...
```

Then follow one of the blob hashes:

```
git cat-file -p <blob-hash>
```

This allows you to move through Git's structure manually:

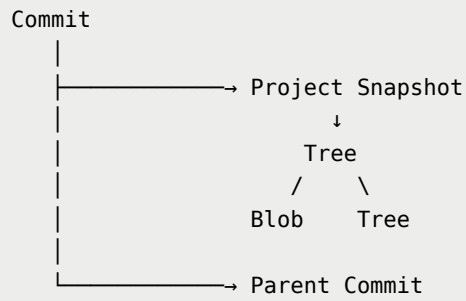
```
Commit
  ↓
Tree
  ↓
Blob
  ↓
File Content
```

33. What Does a Commit Really Represent?

A more accurate definition of a commit is:

A commit object records metadata, points to the root tree representing the project's snapshot, and usually points to one or more parent commits.

Conceptually:



The tree gives the commit its project snapshot.

The parent pointer connects the commit to earlier history.

34. Git History Is a Graph

Suppose the first commit is:

```
Commit A
```

It has no parent.

Then another commit is created:

```
Commit B  
parent → A
```

Then:

```
Commit C  
parent → B
```

Technically, the relationships point backward:

```
C → B → A
```

Each commit knows about its parent.

Humans usually draw the same history chronologically as:

```
A - B - C
```

This graph structure becomes extremely important when learning branches and merges.

For example, a merge commit can have:

```
two parents
```

That is how different lines of development can be connected in Git history.

35. Git Stores Snapshots Without Blindly Duplicating Everything

Git is commonly explained using project snapshots.

That does **not** mean Git blindly copies every file again for every commit.

Suppose Commit A contains:

```
app.txt      → Blob X
README.md    → Blob Y
config.txt   → Blob Z
```

Now only `app.txt` changes.

Commit B may conceptually refer to:

```
app.txt      → Blob X2  changed
README.md    → Blob Y    reused
config.txt   → Blob Z    reused
```

Visualized:

Snapshot A

Tree A

```
|— app.txt → Blob 1
|— README → Blob 2
└— config → Blob 3
```

Snapshot B

Tree B

```
|— app.txt → Blob 4   NEW
|— README → Blob 2   reused
└— config → Blob 3   reused
```

Unchanged content can continue pointing to the same existing objects.

So snapshot-based storage does not mean:

```
Git creates hundreds of complete physical
copies of the entire project for every commit.
```

36. The Staging Area and `.git/index`

The staging area is implemented primarily through:

```
.git/index
```

A useful definition is:

| The index is Git's proposed representation of the contents of the next commit.

The index tells Git which file versions should become part of the next snapshot.

37. Inspecting the Index

Run:

```
git ls-files --stage
```

You may see output like:

```
100644 abc123... 0 app.txt
100644 def456... 0 hello.txt
```

Conceptually:

```
filename
  ↓
object ID
```

The index can represent something like:

For the next commit:

```
app.txt    → Blob X
hello.txt  → Blob Y
```

This is why staging is more than simply marking a file with a flag.

The index is building the content representation that Git can use for the next commit.

38. Putting the Internals Together

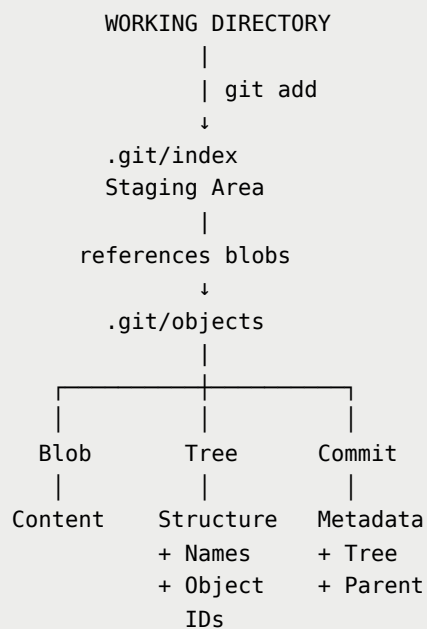
The normal workflow:

```
Working Directory
  ↓ git add
Staging Area / Index
  ↓ git commit
Commit History
```

Internally, the same process can be understood as:

```
File Content
  ↓
Blob Object
  ↓
Index references the Blob
  ↓
Tree represents names and structure
  ↓
Commit points to the Tree
  ↓
Commit points to Parent Commit
  ↓
Git History
```

A more complete mental model is:



39. Important Commands Covered

Repository Setup


```
git init
git status
```

Git Identity

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"

git config user.name
git config user.email
```

Staging and Commits

```
git add <file>
git add file1 file2
git commit -m "Commit message"
```

History

```
git log
git log --oneline
```

Remote Repositories

```
git remote add origin <repository-url>
git remote -v
git push -u origin master
git clone <repository-url>
git fetch
git pull
```

Git Internals

```
ls -la .git
git hash-object <file>
git hash-object -w <file>
git cat-file -t <hash>
git cat-file -p <hash>
git ls-tree HEAD
git ls-tree master
git ls-files --stage
```

40. Final Mental Model

At the beginner level:

```
Edit Files
  ↓
git add
  ↓
Stage Changes
  ↓
git commit
  ↓
Save a Version Locally
  ↓
git push
  ↓
Share It Remotely
```

At the internal level:

```
File Content
  ↓
Blob
  ↓
Tree
  ↓
Commit
  ↓
Parent Commit
```

↓
History Graph

And across local and remote repositories:

```
Working Directory
  ↓ git add
Staging Area
  ↓ git commit
Local Repository
  ↓ git push
Remote Repository

Remote Repository
  ↓ git fetch / git pull
Local Repository
```

Once these mental models are clear, later Git topics such as **branches, merge, rebase, conflicts, reset, revert, and collaborative workflows** become much easier to understand.